

APTITUDEMIND

Adaptive Agentic AI Mentor

POST-CHECKPOINT DEVELOPMENT RECORD

Developer Project	AptitudeMind
Technology	Python + Ollama + Llama 3.2 3B
Development Approach	Local-first Agentic AI
Current Focus	Reliable Question Generation, Validation, Verification and Evaluation

This document is a continuation of the previous "AptitudeMind Complete Development Record" and records the engineering work completed after that checkpoint.

1. Starting Checkpoint

The previous development record placed AptitudeMind at approximately **65% overall completion**. The project already had the following components in place:

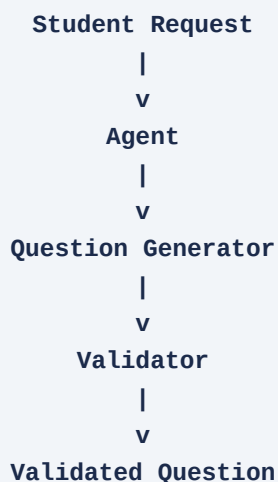
- Local Ollama LLM
- Calculator / tools
- Persistent memory
- Topic system
- Company system
- Question search / question bank
- Adaptive difficulty
- Agent decision layer
- Agent execution layer
- Question generation
- Validator / guardrails
- Partial testing / evaluation

The immediate checkpoint goal was to upgrade the validator with stronger mathematical validation — especially **Algebra / quadratic validation** and **discriminant checking**.

2. Development Completed After the Checkpoint

2.1 Agent – Generator – Validator Integration

The Agent was connected with the question Generator and the Validator. The Agent does not blindly accept an LLM-generated question — invalid generated questions are rejected or regenerated.



2.2 Structured Question Generation

The generated question structure was improved. Generated questions now contain: **question, topic, type, parameters, options, difficulty, company.**

```
{
  "question": "...",
  "topic": "Algebra",
  "type": "QUADRATIC_EQUATION",
  "parameters": {
    "a": 2,
    "b": 5,
    "c": -3
  },
  "options": [
    "1/2",
    "-3/2",
    "-1",
    "3/2"
  ],
  "difficulty": "Hard",
  "company": "TCS"
}
```

2.3 Quadratic Answer Engine

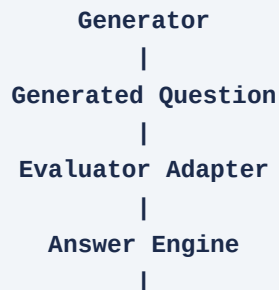
A deterministic quadratic solver was added to the Answer Engine. The system can:

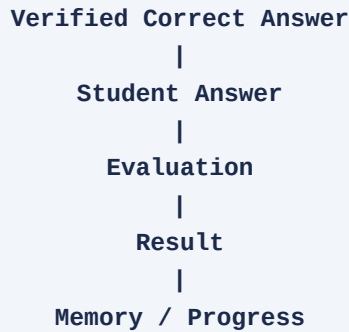
- Read a, b, c parameters
- Calculate the discriminant
- Calculate quadratic roots
- Handle positive / zero / negative discriminants
- Parse numerical options
- Parse fraction options such as $1/2$ and $-3/2$
- Match calculated roots against the provided options
- Return the verified correct option

Important engineering principle: the LLM is not trusted as the mathematical source of truth. Python performs the deterministic calculation.

2.4 Evaluator Integration

The Evaluator was integrated with the Answer Engine.



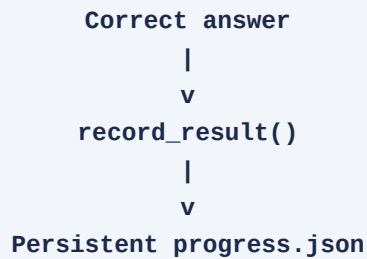


The evaluator:

- Validates question structure
- Adapts nested parameters
- Normalizes question data
- Calls the deterministic Answer Engine
- Determines the verified correct option
- Accepts the student's answer
- Compares student answer with verified answer
- Produces a structured evaluation
- Displays explanation / result

2.5 Progress / Memory Integration

The evaluation system was connected with persistent memory.



The system tracks:

- Total questions
- Correct answers
- Wrong answers
- Topic attempts
- Topic accuracy
- Weak topic

This information is later used by the adaptive difficulty system.

3. Important Bugs Discovered and Fixed

3.1 Quadratic Option Bug

Problem: A generated quadratic question could contain options that did not include the mathematically correct roots.

Example — Equation: $2x^2 + 5x - 3 = 0$

- Correct roots: $x = 1/2$, $x = -3$
- The generated options did not always contain both correct roots.

Fix: A deterministic quadratic solver was added to the Answer Engine. The system now calculates the roots and checks whether the options contain the expected answers.

3.2 Nested Parameters Bug

Problem: Generated questions stored values inside a nested **parameters** object, for example:

```
"parameters": {  
  "cost_price": 800,  
  "profit_percentage": 15  
}
```

But some direct Answer Engine functions expected values in another form.

Fix: The Evaluator now acts as an adapter. It prepares / normalizes the generated question before sending it to the Answer Engine.

3.3 Loss Question Semantic Bug

A very important semantic bug was discovered. The Generator produced:

"A box of 50 pencils was sold for \$60, but the cost price was \$55. What is the loss percentage?"

Parameters: cost_price = 55, selling_price = 60 | Topic: Loss

But Selling Price > Cost Price — therefore this mathematically represents **PROFIT**, not **LOSS**. The old validator allowed the question through.

Fix: Deterministic Profit/Loss semantic validation was added:

Condition	Classification
Selling Price > Cost Price	Profit
Selling Price < Cost Price	Loss
Selling Price = Cost Price	Neither profit nor loss

The validator now rejects mathematically inconsistent Profit/Loss questions.

3.4 Algebra Validation

The validator already contained mathematical validation for Algebra, checking:

- Sum of roots
- Product of roots
- Vieta relationships
- Discriminant
- Quadratic consistency
- Correctness of provided options
- Ambiguous wording

Examples of rejected problems included:

- Incorrect root sum
- Incorrect discriminant
- Options containing no correct root
- Ambiguous "one of the roots" wording

4. Answer Engine Testing

The Answer Engine test suite successfully covered:

- Percentage
- Profit
- Loss
- Average
- Simple Interest
- Distance
- Speed
- Time
- Quadratic Equation

Additional negative tests included:

- Wrong quadratic options
- Wrong percentage options

The Answer Engine successfully returned deterministic calculations and correctly rejected invalid answer-option combinations.

5. Evaluator Testing

The Evaluator test suite completed **14 tests** successfully. Tests included:

- Percentage correct answer
- Percentage incorrect answer
- Profit
- Loss
- Average
- Simple Interest
- Distance
- Speed
- Time
- Quadratic Equation
- Invalid student answer
- Missing question
- Invalid option count
- Unsupported topic

All 14 tests passed.

6. Latest Validator Testing

#	Test Case	Result	Notes
Test 1	Valid Profit Question	VALID	—
Test 2	Invalid Profit Question	INVALID	Correctly rejected — selling price was lower than cost price.
Test 3	Valid Loss Question	VALID	—
Test 4	Invalid Loss Question	INVALID	Correctly rejected — selling price was greater than cost price.
Test 5	Mathematically inconsistent Algebra discriminant	INVALID	—
Test 6	Incorrect Algebra options	INVALID	—
Test 7	Correct Algebra options	VALID	—
Test 8	Mathematically inconsistent Algebra root sum	INVALID	—
Test 9	Valid Percentage Question	VALID	—
Test 10	Invalid Question	INVALID	Errors: missing question, invalid/missing difficulty, missing company info, insufficient options, missing percentage context.
Test 11	Ambiguous Algebra Question	INVALID	Errors: ambiguous "one of the roots" wording, missing unique mathematical identification.

These results demonstrate that the validator is now checking both structural correctness and mathematical consistency.

7. Current AptitudeMind Pipeline



- The LLM generates candidate questions.
- The Validator checks structural and mathematical validity.
- The Answer Engine performs deterministic calculations.
- The Evaluator checks the student's response.
- Memory stores performance.
- Adaptive difficulty uses performance data to select the next question.

8. Files Changed

The following files were modified during this development segment:

File	Role
agent.py	Agent decision and execution logic.
answer_engine.py	Deterministic answer verification and mathematical solving.
evaluator.py	Student-answer evaluation and integration between generated questions and the Answer Engine.
validator.py	Question structure and mathematical validation.

9. GitHub Checkpoint

The Git status showed:

```
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:

  modified:   answer_engine.py
  modified:   evaluator.py
  modified:   validator.py
```

These files were intended to be staged, committed, and pushed as the completed checkpoint.

Note: The final push has not been verified. This record does not claim the push was completed unless a later git status output explicitly confirms it.

10. Current Development Position

AptitudeMind has now progressed from simple LLM question generation toward a more reliable agentic aptitude system.

Completed / Reinforced Layers

- Local LLM
- Tool usage
- Question generation
- Structured question format
- Agent decision / execution
- Mathematical validation
- Deterministic Answer Engine
- Evaluator
- Persistent memory
- Adaptive difficulty foundation
- Profit validation
- Loss validation
- Algebra validation
- Quadratic verification
- Testing

Major Future Layers Remaining

- RAG / knowledge retrieval
- More complete autonomous / adaptive agent workflow
- Backend / API
- Website UI
- Student dashboard
- Practice arena
- AI mentor chat
- Topic library
- Analytics
- Expanded testing / evaluation
- Deployment
- Final documentation
- Final project demo

11. Engineering Principle

BUILD → TEST → DEBUG → CONNECT

AptitudeMind is not being built by blindly trusting the LLM. Instead:

```

      LLM
    -> generates candidate

    Python Validator
    -> checks correctness

    Answer Engine
    -> verifies mathematics

    Evaluator
    -> evaluates student

    Memory
    -> records learning

    Adaptive System
    -> improves the next question
```

This demonstrates reliable Agentic AI engineering.

End of Post-Checkpoint Development Record